Blazor: componenti, rotte e navigazione

Lezione 2 — 3IOT, SST-ICT-01 — Laboratorio guidato: @page, parametri di route, NavMenu, NavigationManager, mini-progetto catalogo

07/10/2026

1 Introduzione in aula: indirizzamento statico e dinamico

Definizione

Ci sono due modi per **indirizzare** le pagine di un sito, cioè per decidere quale **pagina** risponde a un certo **URL**.

- **Indirizzamento statico**: nell'URL compare tutto il **percorso**, strutturato come il **file system** dello spazio web del server che contiene le pagine, e in fondo c'è proprio il **nome del file** chiamato (per esempio `.../prodotto.html`). Entrando nello spazio web del sito, nella parte dove sono salvati i file, si trova **davvero un file** `prodotto.html` (o `.php`: in quel caso dentro c'è un misto di HTML e **PHP**, un linguaggio molto simile al C). Un sito statico navigabile ha tante pagine quanti sono i **link**.
- **Indirizzamento dinamico**: nell'URL **non** compare il nome del file. Il file può chiamarsi in **qualunque modo** (per esempio `Antonio.razor`, o anche `Pippo.razor`), con un **vincolo**: il nome di un componente Razor deve iniziare con la **lettera maiuscola**. Il nome che compare nell'URL lo decide il **programmatore**, con la direttiva `@page`.

Esempio

Sui siti commerciali, nell'URL di un prodotto spesso non c'è un codice o un nome di file, ma una **descrizione leggibile**, con le parole separate da **trattini**: per esempio, nella pagina di un paio di scarpe c'è scritto marca e modello. È pensata per le **persone**, non è un indice tecnico. Con l'indirizzamento statico invece, inviando un form con il metodo **`GET`**, si vede **in chiaro** tutta la composizione della stringa (per esempio `prodotti?id=...`), e dati visibili in chiaro possono dare a **malintenzionati** la possibilità di fare cose particolari. (Attenzione: anche i dati inviati con **`POST`** viaggiano in chiaro se il sito non usa **HTTPS**: la differenza è che con `GET` restano visibili nell'URL, nella **cronologia** del browser e nei **log** del server.)

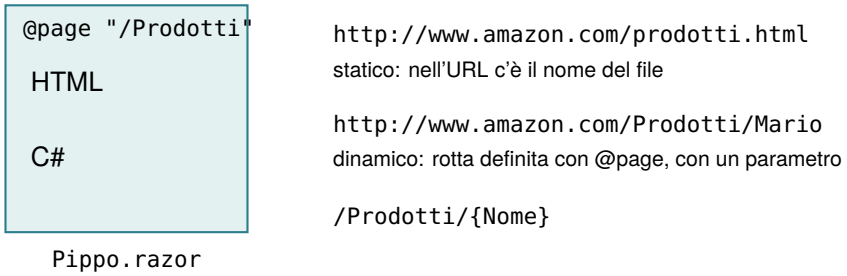


Figura 1: La lavagna della lezione: il file può chiamarsi `Pippo.razor`, ma risponde all'indirizzo scritto in `@page`; a destra un URL statico, uno dinamico e la rotta con il parametro tra graffe.

Nota

Il nome nell'URL funziona come una maschera. Con `@page "/prodotti"` la pagina risponde all'indirizzo `/prodotti`, qualunque sia il nome del file sul disco. Lo slash iniziale indica la **radice** della struttura ad albero delle pagine. In questo modo la **tecnologia** con cui è sviluppata la pagina (C#/Razor, PHP, Java, Ruby…) resta **nascosta all'utente**, e questo riduce le informazioni che un **malintenzionato** può raccogliere sul sistema. È una misura utile ma **non sufficiente da sola**: un'applicazione Blazor si riconosce comunque dai file che il browser scarica, e la **sicurezza** vera viene da **validazione degli input**, **controllo degli accessi** e HTTPS. Tutte le pagine che l'utente deve poter aprire devono avere in testa la direttiva **`@page`** con il loro nome.

Esempio

La personalizzazione del nome (parametro nell'URL). Esempio fatto alla lavagna: una pagina che mostra i **dati** degli allievi della classe. Si chiama la pagina con `/allievi/` seguito dal **nome** dell'allievo. Nel codice C# della pagina viene creata una **variabile** (per esempio `Nome`, o `Studente`) che, nel momento in cui la pagina viene chiamata, viene **valorizzata** con il nome scritto nell'URL. Con quella variabile si può poi fare una **query** sul database, filtrare e recuperare tutti i dati di quella persona (età, dove vive, telefono…) e mostrarli nella pagina. Nella rotta il parametro si scrive **tra graffe**; nel codice si antepone **`[Parameter]`** tra parentesi quadre alla proprietà, per dire che è un parametro da valorizzare con l'URL.

Esempio

La demo del docente: un catalogo letto da database (preparata in circa dieci minuti, con un database di 30 prodotti). La pagina principale (`Home`, rotta `"/"`: quando nell'URL non si mette niente si apre la **home**) contiene un **ciclo** **`@foreach`**: per ogni prodotto letto dal **database** disegna un **box** con la sua classe CSS, il **titolo** (`<h5>`) con il nome del prodotto, un **paragrafo** (`<p>`) con la descrizione, il prezzo e un pulsante. Il pulsante **compone il link** aggiungendo a `/dettaglio/` l'**Id** del prodotto: il primo pulsante chiama `/dettaglio/1`, il secondo `/dettaglio/2` e così via.
L'HTML del box **si scrive una volta sola**: è il **ciclo** a ripeterlo tante volte quanti sono i prodotti. Lato codice, il ciclo **`while (dataReader.Read())`** che legge i dati dal database riempie una **lista**; poi nel frontend il **`@foreach`** scorre la lista e crea un box per **ogni elemento**.

Attenzione

Prevedere tutte le possibilità. Se la pagina ha solo `@page "/prodotti/{Nome}"` e qualcuno la chiama **senza nome** (`/prodotti`), la pagina **non viene trovata**. Un bravo programmatore deve prevederlo, per non far **mai** andare in errore la macchina e per non mostrare **mai** all'utente **messaggi di errore interni**: potrebbero dare informazioni sulla tecnologia o sulla struttura del sito, raccolte poi per violarlo. La soluzione è aggiungere una **seconda direttiva** `@page "/prodotti"`: una pagina può avere **più di una** direttiva `@page`, e risponde a tutti quegli indirizzi. In alternativa si può scrivere una sola direttiva con un **parametro opzionale**, aggiungendo il **punto di domanda**: `@page "/prodotti/{Nome?}"` risponde sia a `/prodotti` sia a `/prodotti/Mario`. Nel codice poi si gestisce il caso in cui il nome **non è valorizzato**, e il caso in cui il nome **non esiste** (per esempio un nome inventato come "Genoveffa"), con un

messaggio chiaro invece di un errore.

Nota

Indicazioni del docente sul documento di laboratorio. Le sezioni che seguono sono una serie di **esercizi** da provare a fare, sei facciate, per capire meglio questi concetti. Due dettagli chiariti in aula:

- la **classe modello** (`Prodotto.cs`) descrive l'**oggetto** da rappresentare, che sia un prodotto o un allievo. Una classe può avere caratteristiche e metodi; per ora solo **proprietà**, senza metodi;
- la cartella **`Models`** va creata nella **radice (root) del progetto**, accanto a `Components`, e non dentro una cartella del database. Il nome `Models` non è obbligatorio, ma è lo **standard**: contiene i **modelli** delle classi degli oggetti da rappresentare.

Attenzione

Non fate copia e incolla. Seguite gli esempi **scrivendo voi il codice**: all'**esame**, fra meno di otto mesi, non avrete il copia e incolla, e più **scrivete a mano** più vi resta in testa.

2 Guida all'esercitazione

Nota

In questo laboratorio pratico si realizza un'**applicazione web** interattiva con **Blazor**. L'obiettivo è capire tre cose:

- come sono strutturati i **componenti Razor**;
- come funziona il sistema di **routing** tramite **URL**;
- come gestire la **navigazione**, sia tramite **menu** sia da **codice** C#.

Ogni passo ha un **tempo indicativo**. I passi segnati come **Sfida express** sono più **difficili**.

3 Cos'è un componente Blazor e la direttiva @page

Definizione

In Blazor l'**interfaccia utente** è composta da **componenti Razor**, cioè file con estensione **`.razor`**. Un componente unisce **HTML** formattato e **logica C#**. Quando un componente deve essere **raggiungibile** direttamente tramite un indirizzo URL nel browser, diventa una **pagina** grazie alla direttiva **`@page "/indirizzo"`**.

Esempio

```
@* File: Components/Pages/Esempio.razor *@
@page "/esempio"

<h3>La mia prima pagina Blazor</h3>
<p>Questo testo e' HTML statico, ma sotto usiamo C#!</p>

<p>Ora attuale del sistema: @OraAttuale</p>

@code {
    private string OraAttuale = DateTime.Now.ToLongTimeString();
}
```

La parte sopra è HTML. **`@OraAttuale`** inserisce nella pagina il **valore** di una variabile C#. Il blocco **`@code { ...}`** contiene la **logica C#** del componente.

Attenzione

Passo 1.1 — Creazione della pagina "Profilo" *Tempo: 20 min*
Obiettivo: creare una **nuova pagina** raggiungibile all'URL `/profilo`.

1. Nella cartella `Components/Pages` **crea** un nuovo file chiamato **`Profilo.razor`**.
2. **Aggiungi** in testa al file la direttiva di **routing** `@page "/profilo"`.
3. **Inserisci** una struttura **HTML** con il tuo nome, un titolo professionale e una **lista** di 3 competenze informatiche.
4. **Avvia** l'applicazione (`F5` oppure `Ctrl+F5`) e **naviga** manualmente all'indirizzo `/profilo`.

4 Parametri di route dinamici (URL dinamici)

Definizione

Spesso una pagina deve mostrare **dati diversi** in base all'URL inserito, per esempio `/profilo/Mario`. In Blazor si passano dati nell'URL definendo una **variabile tra graffe** nella direttiva `@page` e legandola a una **proprietà C#** decorata con l'attributo **`[Parameter]`**. Il nome tra graffe e il nome della proprietà devono **coincidere**.

Esempio

```
@* File: Components/Pages/Profilo.razor *@
@page "/profilo/{Nome}"

<div class="card p-3 my-2">
    <h3>Scheda Utente</h3>

    @if (string.IsNullOrWhiteSpace(Nome))
    {
        <p>Nessun nome utente fornito nell'URL.</p>
```

```
    }
    else
    {
        <h4>Benvenuto, @Nome!</h4>
        <p>Stato Account: <span class="badge bg-success">Attivo</span></p>
    }
</div>

@code {
    [Parameter]
    public string? Nome { get; set; }
}
```

Il **?** dopo `string` indica che la proprietà può anche essere **vuota** (**`null`**). Per questo nel markup si **controlla** con `string.IsNullOrWhiteSpace(Nome)` prima di usarla.

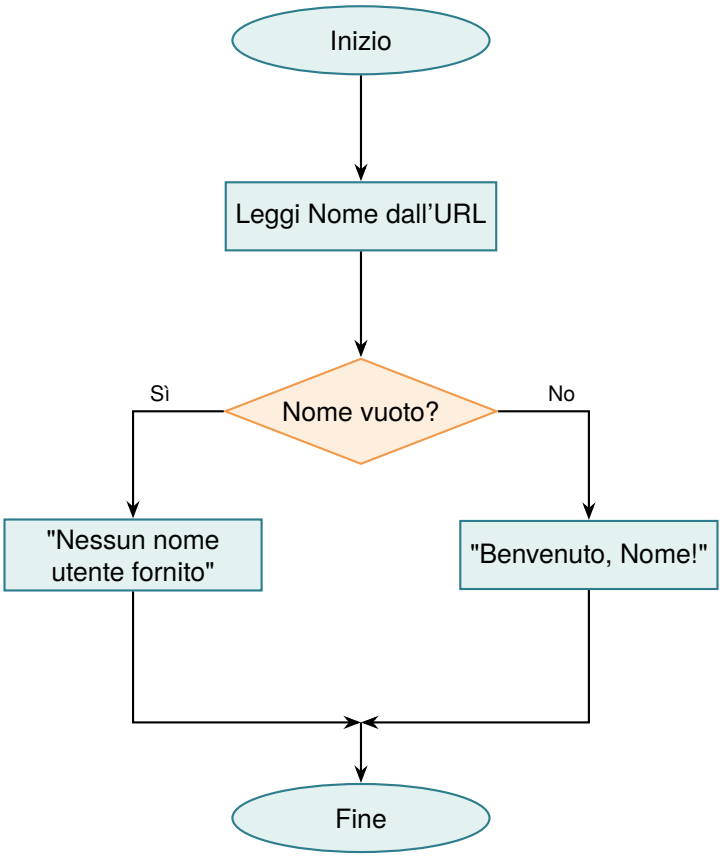


Figura 2: Diagramma di flusso della pagina Profilo: il blocco `@if` decide cosa mostrare in base al parametro ricevuto.

Attenzione

Passo 1.2 — Rendere il profilo dinamico *Tempo: 20 min*

Modifica il file `Profilo.razor` per accettare il **nome dell'utente** direttamente dall'URL:

- **aggiorna** la **rotta** in cima al file: `@page "/profilo/{Nome}"`;
- **dichiara** la proprietà C# con l'attributo `[Parameter]` per **catturare** il valore inserito;
- **testa** nel browser gli URL `/profilo/Mario` e `/profilo/Elena`.

Attenzione

Sfida express 1.3 — Rotta multipla e parametro opzionale *Tempo: 15 min*

Aggiungi una **seconda rotta** in cima a `Profilo.razor`, dichiarando `@page "/profilo"` oltre a quella dinamica: un componente può avere **più direttive** `@page`. Gestisci il caso in cui il valore `Nome` **non sia presente** nell'URL, mostrando la scritta "Utente Anonimo".

```
@page "/profilo"
@page "/profilo/{Nome}"
```

5 Il menu di navigazione: NavMenu e NavLink

Definizione

Per permettere agli utenti di **passare da una pagina all'altra** senza digitare a mano l'URL si usa il **componente menu** dell'applicazione, di solito **`NavMenu.razor`** nella cartella `Layout`. Ogni voce del menu è un componente **`<NavLink>`**: funziona come un **link HTML** `<a>`, ma in più aggiunge da solo la classe CSS **`active`** quando la pagina corrente corrisponde al suo indirizzo, così la voce selezionata appare **evidenziata**.

Esempio

```
<div class="nav-item px-3">
    <NavLink class="nav-link" href="profilo/Studente">
        Profilo
    </NavLink>
</div>
```

Attenzione

Passo 2.1 — Aggiungere la voce al menu *Tempo: 15 min*

1. **Apri** il file `Components/Layout/NavMenu.razor`.
2. **Aggiungi** un elemento `<NavLink>` che punta alla **rotta** `profilo/Studente`.
3. **Verifica** che il click sul menu porti alla pagina e che il pulsante selezionato venga **evidenziato** correttamente.

6 Navigazione programmatica: NavigationManager

Definizione

Quando bisogna **cambiare pagina** come conseguenza di un **evento** C#, per esempio il click su un pulsante o il salvataggio di un form, si usa il servizio **NavigationManager**. Lo si rende disponibile nel componente con la direttiva **`@inject`** e poi si chiama il suo metodo **`Naviga`**

`teTo("/rotta")`. Attenzione: `NavigationManager` **non** è un'estensione del browser, ma un **servizio del framework Blazor** (`Microsoft.AspNetCore.Components`), che il framework fornisce a ogni componente che lo richiede con `@inject`. Non gestisce il menu (quello si fa con `NavLink`): serve a **cambiare pagina da codice** C#, e a leggere l'**indirizzo corrente** (proprietà **`Uri`**).

Esempio

```
@* File: Components/Pages/Home.razor *@
@page "/"
@inject NavigationManager NavManager

<h3>Pannello Principale</h3>

<button class="btn btn-primary me-2" @onclick="VaiAProfiloAdmin">
    Profilo Amministratore
</button>

<button class="btn btn-secondary" @onclick='() => NavigaA("Ospite")'>
    Profilo Ospite
</button>

@code {
    private void VaiAProfiloAdmin()
    {
        NavManager.NavigateTo("/profilo/Admin");
    }

    private void NavigaA(string utente)
    {
        NavManager.NavigateTo($"/profilo/{utente}");
    }
}
```

Il primo pulsante chiama direttamente un **metodo senza parametri**. Il secondo usa una **lambda** (`() => ...`) per poter passare un **argomento** al metodo. Per questo l'attributo è racchiuso tra **apici singoli**: dentro ci sono le virgolette doppie della stringa `"Ospite"`. La stringa `$"/profilo/{utente}"` è una **stringa interpolata** di C#: il **valore** della variabile viene inserito al posto di `{utente}`.

Attenzione

Passo 2.2 — Test del NavigationManager *Tempo: 15 min*

Nella pagina `Profilo.razor` **aggiungi un pulsante** "Torna alla Home" che richiama un **metodo C#** il quale esegue `NavManager.NavigateTo("/")`. Ricorda di aggiungere anche in questa pagina la riga **`@inject NavigationManager NavManager`**: senza, `NavManager` non esiste e il progetto **non compila**.

Attenzione

Sfida express 2.3 — Generatore di profilo casuale *Tempo: 20 min*

Crea una **nuova pagina** `Dashboard.razor` (rotta `/dashboard`) con un pulsante che **estrae a**

caso un nome da un **array** C# e **reindirizza** l’utente alla rotta del profilo corrispondente, via codice.

Esempio

```
@* File: Components/Pages/Dashboard.razor *@
@page "/dashboard"
@inject NavigationManager NavManager

<h3>Dashboard Generatore Profilo</h3>

<button class="btn btn-success" @onclick="GeneraEProfila">
    Genera Utente Casuale
</button>

@code {
    private string[] NomiDisponibili =
        new[] { "Marco", "Sofia", "Luca", "Giulia" };

    private void GeneraEProfila()
    {
        Random rnd = new Random();
        // Next(min, max): max escluso, quindi indice da 0 a Length-1
        int indice = rnd.Next(0, NomiDisponibili.Length);
        string estratto = NomiDisponibili[indice];

        NavManager.NavigateTo($"/profilo/{estratto}");
    }
}
```

Navigazione	Sintassi / strumento	Quando usarla
Dichiarativa	`<NavLink href="rotta">`	**Menu** di navigazione, **link statici** nell’interfaccia HTML.
Programmatica	`NavManager.NavigateTo("/rotta")`	**Eventi** C#, invio di form, **reindirizzamenti condizionali**.

7 Mini-progetto “Catalogo prodotti”

Definizione

Si parte da una **classe modello** C#, che descrive com’è fatto un **prodotto**. Va creata nel file **`Prodotto.cs`**, nella cartella **`Models`**:

```
// File: Models/Prodotto.cs
namespace AppBlazor.Models;

public class Prodotto
{
    public int Id { get; set; }
    public string Nome { get; set; } = "";
    public decimal Prezzo { get; set; }
```

```
    public string Categoria { get; set; } = "";
}
```

decimal è il tipo C# adatto ai **prezzi**, perché non introduce **errori di arrotondamento** come `double`. Nei valori letterali si scrive con il **suffisso** **m**, per esempio `1200.00m`.

Attenzione

Fase 3.1 — Pagina catalogo (`/catalogo`) *Tempo: 20 min*

Crea il file `Components/Pages/Catalogo.razor`:

```
@page "/catalogo"
@using AppBlazor.Models
@inject NavigationManager NavManager

<h3>Catalogo Prodotti</h3>

<div class="row">
    @foreach (var item in ListaProdotti)
    {
        <div class="col-md-4 mb-3">
            <div class="card p-3">
                <h5>@item.Nome</h5>
                <p>Prezzo: EUR @item.Prezzo</p>
                <button class="btn btn-sm btn-outline-primary"
                        @onclick="() => ApriDettaglio(item.Id)">
                    Vedi Dettaglio
                </button>
            </div>
        </div>
    }
</div>

@code {
    private List<Prodotto> ListaProdotti = new()
    {
        new Prodotto { Id = 1, Nome = "Laptop Pro",
                       Prezzo = 1200.00m, Categoria = "PC" },
        new Prodotto { Id = 2, Nome = "Mouse Wireless",
                       Prezzo = 25.50m, Categoria = "Accessori" },
        new Prodotto { Id = 3, Nome = "Tastiera Meccanica",
                       Prezzo = 85.00m, Categoria = "Accessori" }
    };

    private void ApriDettaglio(int id)
    {
        NavManager.NavigateTo($"/dettaglio/{id}");
    }
}
```

Attenzione

Fase 3.2 — Pagina dettaglio (`/dettaglio/{Id:int}`) *Tempo: 25 min*

Crea il file `Components/Pages/DettaglioProdotto.razor`, con un **vincolo** sul parametro. Il suffisso **`:int`** nella rotta fa sì che la pagina risponda solo se dopo `/dettaglio/` c’è un **numero**

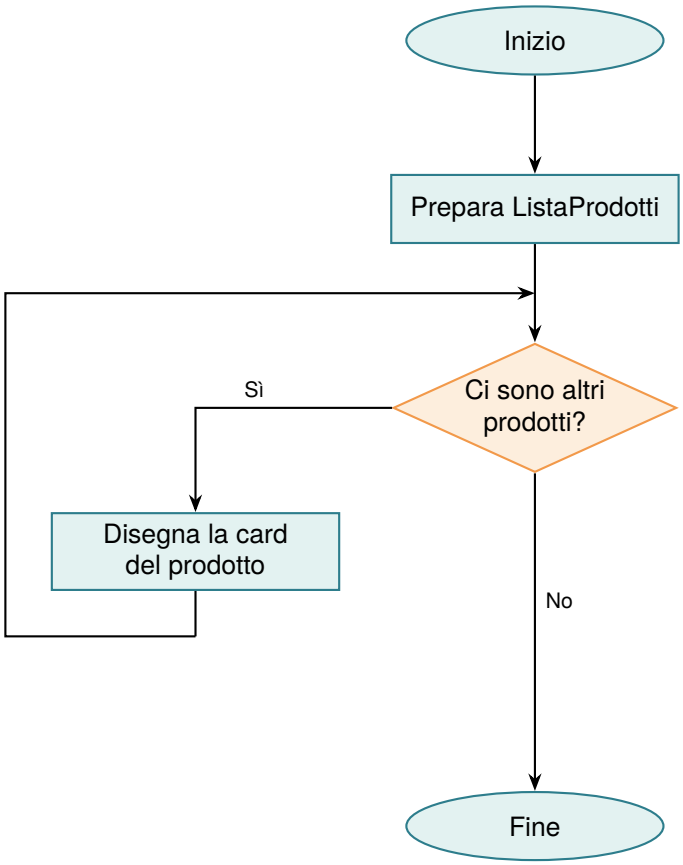


Figura 3: Diagramma di flusso del ciclo `@foreach` della pagina Catalogo: si disegna una card per ogni prodotto della lista, poi si esce dal ciclo.

intero: con `/dettaglio/abc` la rotta **non viene trovata**.

```
@* File: Components/Pages/DettaglioProdotto.razor *@
@page "/dettaglio/{Id:int}"
@using AppBlazor.Models
@inject NavigationManager NavManager

@if (ProdottoTrovato == null)
{
    <div class="alert alert-danger">Prodotto non trovato (ID: @Id)!</div>
}
else
{
    <div class="card p-4">
        <h2>@ProdottoTrovato.Nome</h2>
        <p>Categoria: @ProdottoTrovato.Categoria</p>
        <h4 class="text-success">Prezzo: EUR @ProdottoTrovato.Prezzo</h4>
    </div>
}

<button class="btn btn-secondary mt-3" @onclick="TornaAlCatalogo">
    Torna al Catalogo
</button>

@code {
    [Parameter]
```

```
    public int Id { get; set; }

    private Prodotto? ProdottoTrovato;

    protected override void OnInitialized()
    {
        // Simula la ricerca nell'elenco
        var lista = new List<Prodotto>
        {
            new Prodotto { Id = 1, Nome = "Laptop Pro",
                       Prezzo = 1200.00m, Categoria = "PC" },
            new Prodotto { Id = 2, Nome = "Mouse Wireless",
                       Prezzo = 25.50m, Categoria = "Accessori" },
            new Prodotto { Id = 3, Nome = "Tastiera Meccanica",
                       Prezzo = 85.00m, Categoria = "Accessori" }
        };

        // primo prodotto con quell'Id, oppure null se non esiste
        ProdottoTrovato = lista.FirstOrDefault(p => p.Id == Id);
    }

    private void TornaAlCatalogo()
    {
        NavManager.NavigateTo("/catalogo");
    }
}
```

Nota

`OnInitialized()` è un metodo del **ciclo di vita** del componente: Blazor lo esegue **una volta**, quando il componente viene creato, dopo aver ricevuto i **parametri** dall'URL. `FirstOrDefault(p => p.Id == Id)` restituisce il **primo prodotto** con quell'`Id`, oppure `null` se non esiste: è proprio il caso gestito dal ramo `@if (ProdottoTrovato == null)`, che evita il **crash** dell'applicazione con un ID inesistente.

Esempio

Task bonus: crea un **componente riutilizzabile** `SchedaProdotto.razor` che riceve come `[Parameter]` un **oggetto** `Prodotto`, e usalo all'interno del ciclo `@foreach` della pagina Catalogo al posto del **markup** della card. Un componente **senza** `@page` non è una pagina: non ha un URL, si usa come un **tag**, per esempio `<SchedaProdotto Prodotto="item" />`.

8 Checklist di autovalutazione

- ☐ Tutte le pagine hanno la **direttiva** `@page` corretta in prima riga.
- ☐ I **link del menu** funzionano e mostrano correttamente lo **stato attivo**.
- ☐ La **navigazione** con **parametro dinamico** mostra i dati corretti.
- ☐ La ricerca di un **ID inesistente** viene gestita **senza crash** dell'applicazione.

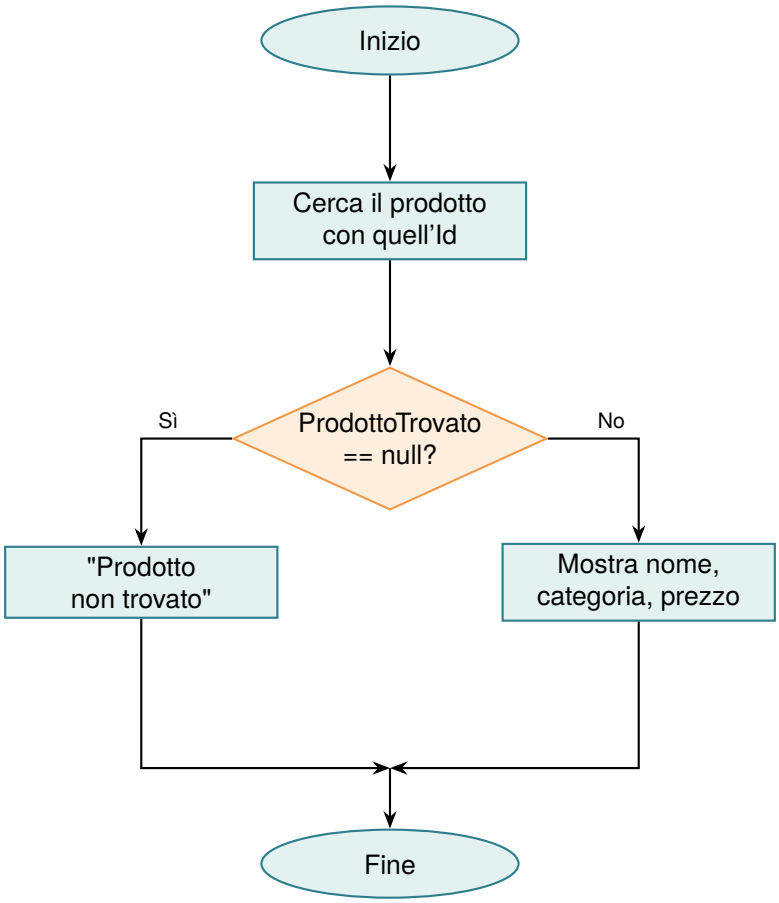


Figura 4: Diagramma di flusso della pagina Dettaglio: un ID inesistente viene gestito con un messaggio, senza far andare in crash l'applicazione.

9 Cosa ricordare di questa lezione

- Un **componente Razor** (`.razor`) unisce HTML e codice C#; con **`@page "/indirizzo"`** diventa una **pagina** raggiungibile da URL.
- I **parametri di route** si dichiarano **tra graffe** nella rotta (`{Nome}`) e si ricevono in una proprietà con **`[Parameter]`** dallo stesso nome. Un **vincolo** come `{Id:int}` accetta **solo numeri interi**.
- Un componente può avere **più direttive** `@page`, per esempio con e senza parametro.
- Navigazione **dichiarativa**: **`<NavLink>`** nel menu, che **evidenzia da solo** la voce attiva.
- Navigazione **programmatica**: **`@inject NavigationManager`** e poi **`NavigateTo("/rotta")`**, da usare quando serve **cambiare pagina** dopo un **evento** C#.
- Il caso "**dato non trovato**" va **sempre** gestito, senza dare per scontato che esista.